

CSE 344 System Programming

Homework 6 Report

Magic Academy

Name: Halit Batuhan Aslan
Student ID: 220104004003
Course: CSE 344 System Programming
Homework: 6
Date: 14 May 2026

Contents

1	Introduction	2
2	System Design	2
2.1	No Thread Per Client	2
2.2	Capacity Handling	3
2.3	Idle Timeout Handling	3
3	Program Structure	3
4	Data Structures	3
5	Protocol Handling	4
5.1	Ingredient Updates	4
6	Partial Read and Write Handling	4
7	Signal Handling and Shutdown	5
8	Output Format	5
9	Build Result	5
10	Mandatory Test Scenarios	6
10.1	Scenario 1	6
10.2	Scenario 2	7
10.3	Scenario 3	9
10.4	Scenario 4	11
11	Challenges	14
11.1	Handling TCP Lines Correctly	14
11.2	Idle Timeout Without Sleep	14
11.3	Shared Client Code	14
12	Conclusion	14

1 Introduction

This homework implements a TCP based Magic Academy system in C. There are three separate programs: `hogwarts`, `wizard`, and `professor`. The server accepts both wizard and professor clients on the same TCP port. It uses `select()` for all connected clients, so there is no thread-per-client model.

The implementation is split into small modules. The `hogwarts.c`, `wizard.c`, and `professor.c` files only contain program entry points. Most server logic is moved to `server.c`, `server_clients.c`, `server_protocol.c`, `server_options.c`, and `server_log.c`. Common socket and parsing helpers are in `common.c`. Ingredient loading and snapshot creation are in `ingredients.c`. This made the code easier to test and also keeps the three executable targets separate.

2 System Design

The server starts by parsing command line options, loading the ingredient file, opening the log file, and creating one listening TCP socket. After this setup, the server enters one main event loop. This loop manages both the listening socket and all client sockets with `select()`.

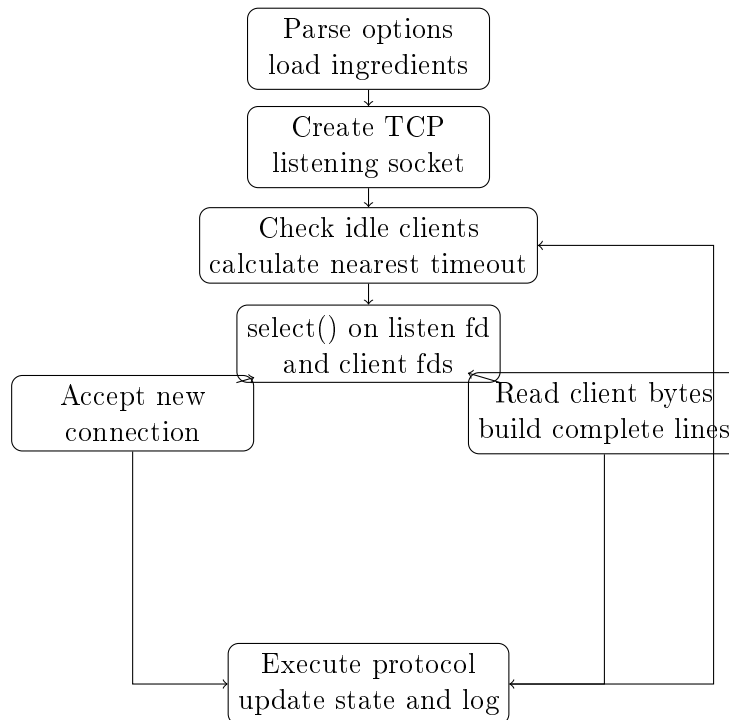


Figure 1: Single-threaded server loop

2.1 No Thread Per Client

The server does not create threads or child processes for clients. Each connected client has a record inside the server context. The main loop rebuilds the `fd_set` before every `select()` call. If the listening socket is readable, the server calls `accept()`. If a client socket is readable, the server reads bytes from that socket and processes complete protocol lines.

2.2 Capacity Handling

The maximum number of clients is given by `-n`. When this limit is reached, the server still accepts the new TCP connection first. It immediately sends:

```
ERR HOGWARTS_FULL
```

Then it logs the rejection and closes that new socket. This is done so the connecting client receives a clear error instead of waiting silently.

2.3 Idle Timeout Handling

Every client stores `last_active`. The server checks all clients before each `select()` call. If a client has been inactive for at least the configured timeout, the server sends:

```
TIMEOUT DISCONNECT
```

Then the client state is cleaned. For remaining clients, the server calculates the minimum remaining time and passes it as the timeout argument to `select()`. Because of this, timeout control uses no `sleep()` and no helper thread in the server.

3 Program Structure

File	Responsibility
<code>hogwarts.c</code>	Server executable entry point. Calls <code>run_hogwarts_server</code> .
<code>server.c</code> / <code>server.h</code>	Main server lifecycle, <code>select()</code> loop, timeout calculation, shutdown sequence.
<code>server_types.h</code>	Shared server structs such as <code>client_t</code> , <code>server_options_t</code> , and <code>server_context_t</code> .
<code>server_clients.c</code> / <code>.h</code>	Listening socket, accepting clients, full server rejection, client cleanup.
<code>server_protocol.c</code> / <code>.h</code>	Protocol commands: <code>ENROLL</code> , <code>BREW</code> , <code>CONSUME</code> , <code>SPELLBOOK</code> , <code>INSPECT</code> , <code>SCROLL</code> , <code>ROSTER</code> , <code>APPARATE</code> .
<code>server_options.c</code> / <code>.h</code>	Command line parsing and usage message.
<code>server_log.c</code> / <code>.h</code>	Timestamped server logging to stdout and the given log file.
<code>wizard.c</code>	Wizard executable entry point.
<code>professor.c</code>	Professor executable entry point.
<code>client_app.c</code> / <code>.h</code>	Shared TCP client loop used by both wizard and professor.
<code>common.c</code> / <code>.h</code>	Shared helpers for sending complete lines, parsing numbers, splitting commands.
<code>ingredients.c</code> / <code>.h</code>	Ingredient table loading, lookup, scroll snapshot, spellbook snapshot.

4 Data Structures

The most important server-side structure is `client_t`. It stores all per-client state.

```
typedef struct
{
    int fd;
```

```

char username[MAX_USERNAME_LENGTH + 1];
char type[MAX_TYPE_LENGTH + 1];
char line_buf[MAX_LINE_LENGTH];
size_t line_len;
int line_too_long;
time_t last_active;
long *spellbook;
} client_t;

```

The spellbook is a `long` array. It has the same order and size as the global ingredient table. For example, if `MOONSTONE` is at index 0 in the global ingredient table, the wizard's amount of `MOONSTONE` is stored at `spellbook[0]`. This avoids another map structure and is simple for this homework.

The global ingredient list is stored in `ingredient_table_t`. Invalid ingredient file lines are skipped silently as required. Duplicate ingredient names are also skipped.

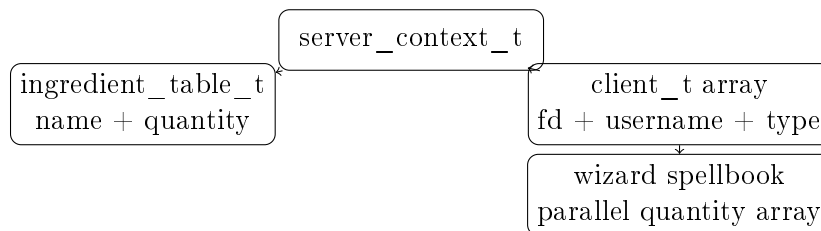


Figure 2: Main server data ownership

5 Protocol Handling

The server requires `ENROLL` as the first command. If a client sends any other command before enrolling, the server responds with:

```
ERR NOT_ENROLLED
```

After enrollment, the server uses the stored client type to authorize commands. Wizards can use `BREW`, `CONSUME`, and `SPELLBOOK`. Professors can use `INSPECT`, `SCROLL`, and `ROSTER`. If a professor sends `BREW` or `CONSUME`, the server returns `ERR UNAUTHORIZED`. The same authorization rule is applied when a wizard tries professor commands.

5.1 Ingredient Updates

`BREW` increases both the global ingredient quantity and the wizard's spellbook quantity. `CONSUME` decreases both, but only if the global stock and the wizard's own spellbook have enough amount. The global quantity never becomes negative.

6 Partial Read and Write Handling

TCP does not preserve message boundaries. Because of that, the server stores incoming bytes in `line_buf`. When a newline is received, that complete line is parsed. If more than 511 useful characters arrive before newline, `line_too_long` is marked and the server answers `ERR TOOLONG` when the newline is reached.

Outgoing messages use `send_all`. It keeps calling `send()` until all bytes are written. This is needed because `send()` may write only part of a message.

7 Signal Handling and Shutdown

The server and clients install a `SIGINT` handler. The handler only sets a volatile `sig_atomic_t` flag. The real shutdown logic happens in the normal control flow.

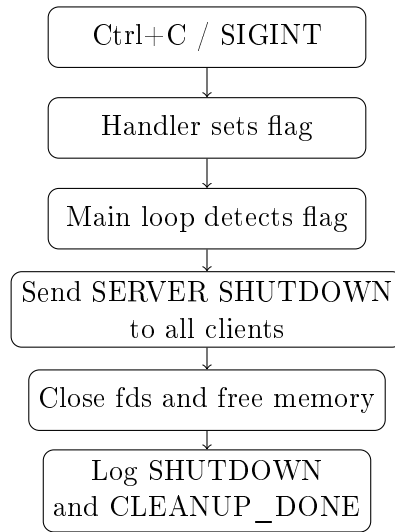


Figure 3: Server shutdown sequence

For wizard and professor clients, Ctrl+C causes the main loop to send `APPARATE`. The client then waits for `OK APPARATE` or a shutdown/timeout notice, closes the socket, and exits.

8 Output Format

Server log lines use the required prefix style with timestamps:

```
[2026-05-14 21:47:00] [SERVER] CLEANUP_DONE clients=0
```

Client outputs use:

```
[WIZARD harry] RECEIVED OK SPELLBOOK MOONSTONE:10  
[PROFESSOR mcgonagall] RECEIVED ERR UNAUTHORIZED
```

The assignment examples show some client lines without the bracket prefix, but the output keyword table says the prefix is mandatory. So I used the stricter table format.

9 Build Result

```
halit@debian:~/MyHw/Homework6$ make  
gcc -Wall -Wextra -std=c11 -c hogwarts.c  
gcc -Wall -Wextra -std=c11 -c server.c  
gcc -Wall -Wextra -std=c11 -c server_options.c  
gcc -Wall -Wextra -std=c11 -c server_log.c  
gcc -Wall -Wextra -std=c11 -c server_clients.c  
gcc -Wall -Wextra -std=c11 -c server_protocol.c  
gcc -Wall -Wextra -std=c11 -c common.c  
gcc -Wall -Wextra -std=c11 -c ingredients.c  
gcc -Wall -Wextra -std=c11 -o hogwarts.o server.o server_options.o server_log.o server_clients.o server_protocol.o common.o ingredients.o  
gcc -Wall -Wextra -std=c11 -c wizard.c  
gcc -Wall -Wextra -std=c11 -c client_app.c  
gcc -Wall -Wextra -std=c11 -o wizard.o wizard.o client_app.o common.o  
gcc -Wall -Wextra -std=c11 -c professor.c  
gcc -Wall -Wextra -std=c11 -o professor.o professor.o client_app.o common.o  
halit@debian:~/MyHw/Homework6$
```

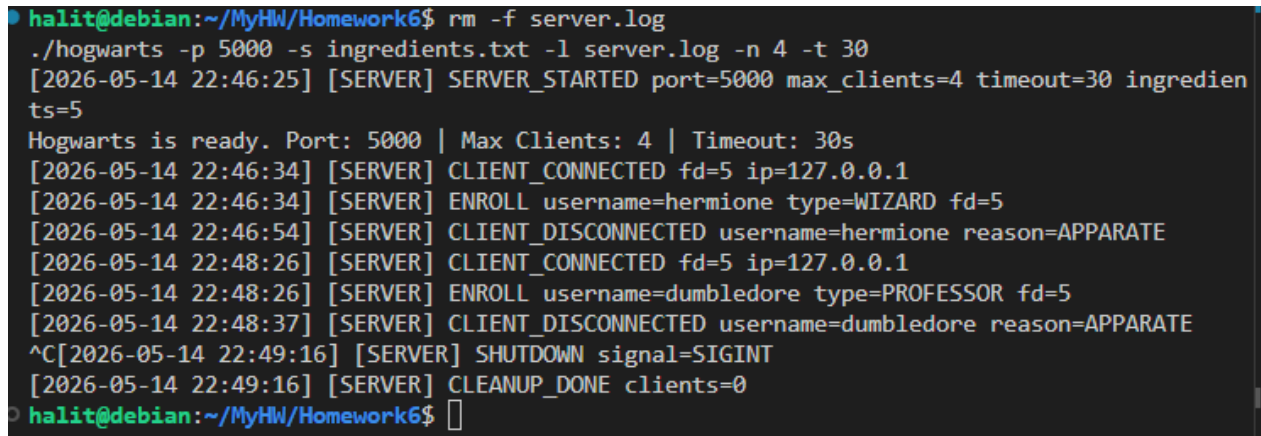
Figure 4: Build output with gcc flags and make clean

10 Mandatory Test Scenarios

The following figures are uncut terminal screenshots taken during the mandatory scenario runs. Each scenario was started with a fresh server, and the screenshots show the commands and the program outputs from start to finish.

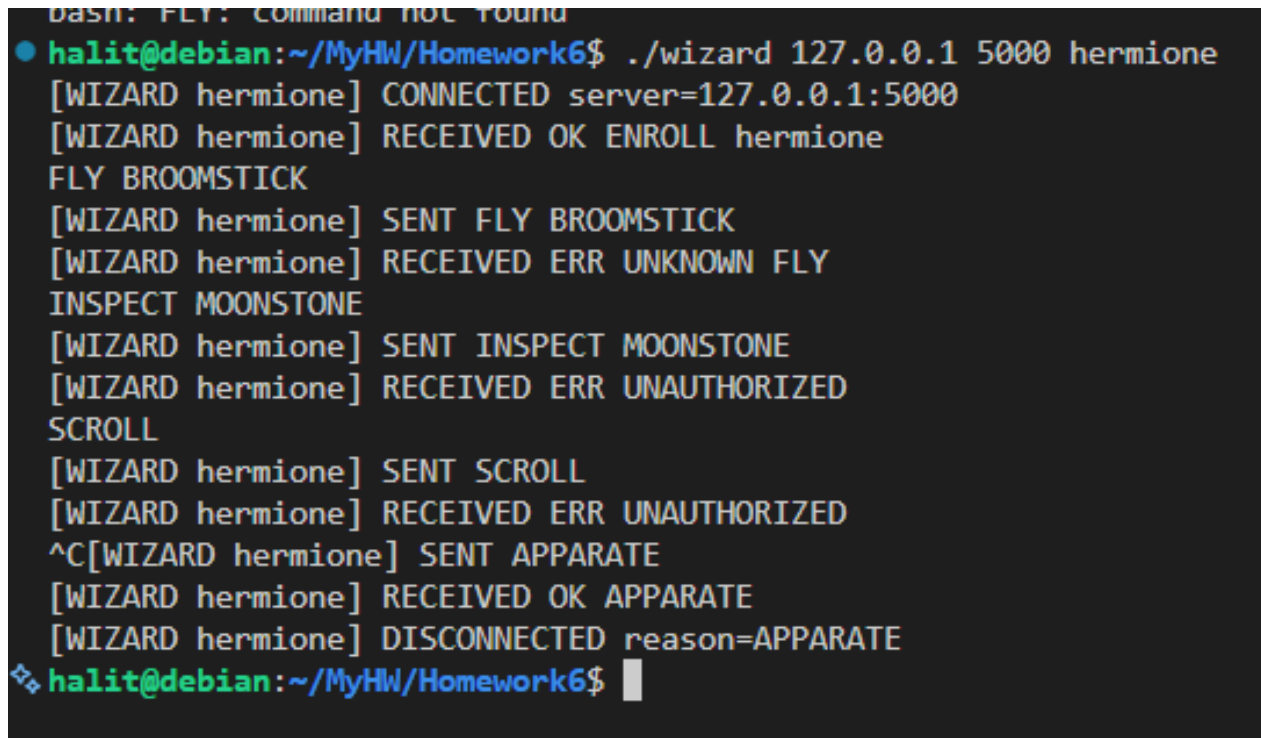
10.1 Scenario 1

This scenario checks duplicate username, command before ENROLL, unknown wizard command, unauthorized professor commands, unauthorized wizard professor-commands, and normal disconnect.



```
halit@debian:~/MyHW/Homework6$ rm -f server.log
./hogwarts -p 5000 -s ingredients.txt -l server.log -n 4 -t 30
[2026-05-14 22:46:25] [SERVER] SERVER_STARTED port=5000 max_clients=4 timeout=30 ingredien
ts=5
Hogwarts is ready. Port: 5000 | Max Clients: 4 | Timeout: 30s
[2026-05-14 22:46:34] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-14 22:46:34] [SERVER] ENROLL username=hermione type=WIZARD fd=5
[2026-05-14 22:46:54] [SERVER] CLIENT_DISCONNECTED username=hermione reason=APPARATE
[2026-05-14 22:48:26] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-14 22:48:26] [SERVER] ENROLL username=dumbledore type=PROFESSOR fd=5
[2026-05-14 22:48:37] [SERVER] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
^C[2026-05-14 22:49:16] [SERVER] SHUTDOWN signal=SIGINT
[2026-05-14 22:49:16] [SERVER] CLEANUP_DONE clients=0
halit@debian:~/MyHW/Homework6$
```

Figure 5: Scenario 1 server terminal and log



```
DASH: FLY: Command not found
halit@debian:~/MyHW/Homework6$ ./wizard 127.0.0.1 5000 hermione
[WIZARD hermione] CONNECTED server=127.0.0.1:5000
[WIZARD hermione] RECEIVED OK ENROLL hermione
FLY BROOMSTICK
[WIZARD hermione] SENT FLY BROOMSTICK
[WIZARD hermione] RECEIVED ERR UNKNOWN FLY
INSPECT MOONSTONE
[WIZARD hermione] SENT INSPECT MOONSTONE
[WIZARD hermione] RECEIVED ERR UNAUTHORIZED
SCROLL
[WIZARD hermione] SENT SCROLL
[WIZARD hermione] RECEIVED ERR UNAUTHORIZED
^C[WIZARD hermione] SENT APPARATE
[WIZARD hermione] RECEIVED OK APPARATE
[WIZARD hermione] DISCONNECTED reason=APPARATE
halit@debian:~/MyHW/Homework6$
```

Figure 6: Scenario 1 wizard Hermione terminal

```
halit@debian:~/MyHW/Homework6$ ./wizard 127.0.0.1 5000 hermione
[WIZARD hermione] CONNECTED server=127.0.0.1:5000
[WIZARD hermione] RECEIVED OK ENROLL hermione
APPARATE
[WIZARD hermione] SENT APPARATE
[WIZARD hermione] RECEIVED OK APPARATE
[WIZARD hermione] DISCONNECTED reason=APPARATE
halit@debian:~/MyHW/Homework6$
```

Figure 7: Scenario 1 duplicate Hermione terminal

```
halit@debian:~/MyHW/Homework6$ nc 127.0.0.1 5000
ROSTER
ERR NOT_ENROLLED

```

Figure 8: Scenario 1 raw client before ENROLL

```
halit@debian:~/MyHW/Homework6$ ./professor 127.0.0.1 5000 dumbledore
[PROFESSOR dumbledore] CONNECTED server=127.0.0.1:5000
[PROFESSOR dumbledore] RECEIVED OK ENROLL dumbledore
BREW MOONSTONE 5
[PROFESSOR dumbledore] SENT BREW MOONSTONE 5
[PROFESSOR dumbledore] RECEIVED ERR UNAUTHORIZED
CONSUME MOONSTONE 5
[PROFESSOR dumbledore] SENT CONSUME MOONSTONE 5
[PROFESSOR dumbledore] RECEIVED ERR UNAUTHORIZED
^C[PROFESSOR dumbledore] SENT APPARATE
[PROFESSOR dumbledore] RECEIVED OK APPARATE
[PROFESSOR dumbledore] DISCONNECTED reason=APPARATE
halit@debian:~/MyHW/Homework6$
```

Figure 9: Scenario 1 professor Dumbledore terminal

10.2 Scenario 2

This scenario checks ingredient updates, spellbook tracking, insufficient own spellbook amount, unknown ingredient, scroll snapshot, roster, and professor inspect.


```

halit@debian:~/MyHW/Homework6$ ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 4 -t 30
[2026-05-14 23:01:17] [SERVER] SERVER_STARTED port=5000 max_clients=4 timeout=30 ingredients=5
Hogwarts is ready. Port: 5000 | Max Clients: 4 | Timeout: 30s
[2026-05-14 23:01:20] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-14 23:01:20] [SERVER] ENROLL username=harry type=WIZARD fd=5
[2026-05-14 23:01:23] [SERVER] BREW wizard=harry ingredient=MOONSTONE qty=10 old_qty=100 new_qty=110
[2026-05-14 23:01:30] [SERVER] BREW wizard=harry ingredient=UNICORN_HAIR qty=15 old_qty=20 new_qty=35
[2026-05-14 23:01:36] [SERVER] CONSUME wizard=harry ingredient=MOONSTONE qty=5 old_qty=110 new_qty=105
[2026-05-14 23:01:49] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-14 23:01:49] [SERVER] ENROLL username=mcgonagall type=PROFESSOR fd=6
[2026-05-14 23:01:52] [SERVER] INSPECT professor=mcgonagall ingredient=DRAGON_SCALE qty=50
[2026-05-14 23:01:55] [SERVER] SCROLL professor=mcgonagall ingredients=5
[2026-05-14 23:01:59] [SERVER] ROSTER professor=mcgonagall clients=2
[2026-05-14 23:02:08] [SERVER] CLIENT_DISCONNECTED username=harry reason=APPARATE
[2026-05-14 23:02:11] [SERVER] CLIENT_DISCONNECTED username=mcgonagall reason=APPARATE
^C[2026-05-14 23:02:21] [SERVER] SHUTDOWN signal=SIGINT
[2026-05-14 23:02:21] [SERVER] CLEANUP_DONE clients=0
halit@debian:~/MyHW/Homework6$

```

Figure 10: Scenario 2 server terminal and log

```

halit@debian:~/MyHW/Homework6$ ./wizard 127.0.0.1 5000 harry
[WIZARD harry] CONNECTED server=127.0.0.1:5000
[WIZARD harry] RECEIVED OK ENROLL harry
BREW MOONSTONE 10
[WIZARD harry] SENT BREW MOONSTONE 10
[WIZARD harry] RECEIVED OK BREW MOONSTONE 10 110
SPELLBOOK
[WIZARD harry] SENT SPELLBOOK
[WIZARD harry] RECEIVED OK SPELLBOOK MOONSTONE:10
BREW UNICORN_HAIR 15
[WIZARD harry] SENT BREW UNICORN_HAIR 15
[WIZARD harry] RECEIVED OK BREW UNICORN_HAIR 15 35
SPELLBOOK
[WIZARD harry] SENT SPELLBOOK
[WIZARD harry] RECEIVED OK SPELLBOOK MOONSTONE:10,UNICORN_HAIR:15
CONSUME MOONSTONE 5
[WIZARD harry] SENT CONSUME MOONSTONE 5
[WIZARD harry] RECEIVED OK CONSUME MOONSTONE 5 105
CONSUME UNICORN_HAIR 20
[WIZARD harry] SENT CONSUME UNICORN_HAIR 20
[WIZARD harry] RECEIVED ERR INSUFFICIENT_INGREDIENTS
BREW BUTTERBEER 5
[WIZARD harry] SENT BREW BUTTERBEER 5
[WIZARD harry] RECEIVED ERR UNKNOWN_INGREDIENT
^C[WIZARD harry] SENT APPARATE
[WIZARD harry] RECEIVED OK APPARATE
[WIZARD harry] DISCONNECTED reason=APPARATE
halit@debian:~/MyHW/Homework6$

```

Figure 11: Scenario 2 wizard Harry terminal

```

halit@debian:~/MyHW/Homework6$ ./professor 127.0.0.1 5000 mcgonagall
[PROFESSOR mcgonagall] CONNECTED server=127.0.0.1:5000
[PROFESSOR mcgonagall] RECEIVED OK ENROLL mcgonagall
INSPECT DRAGON_SCALE
[PROFESSOR mcgonagall] SENT INSPECT DRAGON_SCALE
[PROFESSOR mcgonagall] RECEIVED OK INSPECT DRAGON_SCALE 50
SCROLL
[PROFESSOR mcgonagall] SENT SCROLL
[PROFESSOR mcgonagall] RECEIVED OK SCROLL MOONSTONE:105,DRAGON_SCALE:50,PHOENIX_FEATHER:30,MANDRAKE_ROOT:75,UNICORN_HAIR:35
ROSTER
[PROFESSOR mcgonagall] SENT ROSTER
[PROFESSOR mcgonagall] RECEIVED OK ROSTER harry,mcgonagall
INSPECT BUTTERBEER
[PROFESSOR mcgonagall] SENT INSPECT BUTTERBEER
[PROFESSOR mcgonagall] RECEIVED ERR UNKNOWN_INGREDIENT
^C[PROFESSOR mcgonagall] SENT APPARATE
[PROFESSOR mcgonagall] RECEIVED OK APPARATE
[PROFESSOR mcgonagall] DISCONNECTED reason=APPARATE
halit@debian:~/MyHW/Homework6$ 

```

Figure 12: Scenario 2 professor McGonagall terminal

10.3 Scenario 3

This scenario checks maximum capacity, rejection recovery after a client leaves, and roster after a new client connects.

```

halit@debian:~/MyHW/Homework6$ ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 3 -t 60
[2026-05-14 23:14:47] [SERVER] SERVER_STARTED port=5000 max_clients=3 timeout=60 ingredients=5
Hogwarts is ready. Port: 5000 | Max Clients: 3 | Timeout: 60s
[2026-05-14 23:14:50] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-14 23:14:50] [SERVER] ENROLL username=harry type=WIZARD fd=5
[2026-05-14 23:14:55] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-14 23:14:55] [SERVER] ENROLL username=ron type=WIZARD fd=6
[2026-05-14 23:15:00] [SERVER] CLIENT_CONNECTED fd=7 ip=127.0.0.1
[2026-05-14 23:15:00] [SERVER] ENROLL username=dumbledore type=PROFESSOR fd=7
[2026-05-14 23:15:06] [SERVER] REJECTED fd=8 ip=127.0.0.1 reason=HOGWARTS_FULL
[2026-05-14 23:15:11] [SERVER] REJECTED fd=8 ip=127.0.0.1 reason=HOGWARTS_FULL
[2026-05-14 23:15:17] [SERVER] CLIENT_DISCONNECTED username=harry reason=APPARATE
[2026-05-14 23:15:23] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-14 23:15:23] [SERVER] ENROLL username=neville type=WIZARD fd=5
[2026-05-14 23:15:27] [SERVER] BREW wizard=neville ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105
[2026-05-14 23:15:38] [SERVER] ROSTER professor=dumbledore clients=3
[2026-05-14 23:15:47] [SERVER] CLIENT_DISCONNECTED username=ron reason=APPARATE
[2026-05-14 23:15:51] [SERVER] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[2026-05-14 23:15:56] [SERVER] CLIENT_DISCONNECTED username=neville reason=APPARATE
^C[2026-05-14 23:16:03] [SERVER] SHUTDOWN signal=SIGINT
[2026-05-14 23:16:03] [SERVER] CLEANUP_DONE clients=0
halit@debian:~/MyHW/Homework6$ 

```

Figure 13: Scenario 3 server terminal and log

```

halit@debian:~/MyHW/Homework6$ ./wizard 127.0.0.1 5000 harry
[WIZARD harry] CONNECTED server=127.0.0.1:5000
[WIZARD harry] RECEIVED OK ENROLL harry
^C[WIZARD harry] SENT APPARATE
[WIZARD harry] RECEIVED OK APPARATE
[WIZARD harry] DISCONNECTED reason=APPARATE
halit@debian:~/MyHW/Homework6$ 

```

Figure 14: Scenario 3 wizard Harry terminal

```

halit@debian:~/MyHW/Homework6$ ./wizard 127.0.0.1 5000 ron
[WIZARD ron] CONNECTED server=127.0.0.1:5000
[WIZARD ron] RECEIVED OK ENROLL ron
^C[WIZARD ron] SENT APPARATE
[WIZARD ron] RECEIVED OK APPARATE
[WIZARD ron] DISCONNECTED reason=APPARATE
halit@debian:~/MyHW/Homework6$ 

```

Figure 15: Scenario 3 wizard Ron terminal

```

halit@debian:~/MyHW/Homework6$ ./professor 127.0.0.1 5000 dumbledore
[PROFESSOR dumbledore] CONNECTED server=127.0.0.1:5000
[PROFESSOR dumbledore] RECEIVED OK ENROLL dumbledore
ROSTER
[PROFESSOR dumbledore] SENT ROSTER
[PROFESSOR dumbledore] RECEIVED OK ROSTER neville,ron,dumbledore
^C[PROFESSOR dumbledore] SENT APPARATE
[PROFESSOR dumbledore] RECEIVED OK APPARATE
[PROFESSOR dumbledore] DISCONNECTED reason=APPARATE
halit@debian:~/MyHW/Homework6$ 

```

Figure 16: Scenario 3 professor Dumbledore terminal

```

halit@debian:~/MyHW/Homework6$ ./wizard 127.0.0.1 5000 voldemort
[WIZARD voldemort] CONNECTED server=127.0.0.1:5000
[WIZARD voldemort] RECEIVED ERR HOGWARTS_FULL
[WIZARD voldemort] DISCONNECTED reason=hangup
halit@debian:~/MyHW/Homework6$ 

```

Figure 17: Scenario 3 rejected wizard Voldemort terminal

```

halit@debian:~/MyHW/Homework6$ ./professor 127.0.0.1 5000 snape
[PROFESSOR snape] CONNECTED server=127.0.0.1:5000
[PROFESSOR snape] RECEIVED ERR HOGWARTS_FULL
[PROFESSOR snape] DISCONNECTED reason=hangup
halit@debian:~/MyHW/Homework6$ 

```

Figure 18: Scenario 3 rejected professor Snape terminal

```
halit@debian:~/MyHW/Homework6$ ./wizard 127.0.0.1 5000 neville
[WIZARD neville] CONNECTED server=127.0.0.1:5000
[WIZARD neville] RECEIVED OK ENROLL neville
BREW MOONSTONE 5
[WIZARD neville] SENT BREW MOONSTONE 5
[WIZARD neville] RECEIVED OK BREW MOONSTONE 5 105
^C[WIZARD neville] SENT APPARATE
[WIZARD neville] RECEIVED OK APPARATE
[WIZARD neville] DISCONNECTED reason=APPARATE
halit@debian:~/MyHW/Homework6$
```

Figure 19: Scenario 3 wizard Neville terminal

10.4 Scenario 4

This scenario checks idle timeout, continuing activity for another wizard, later client joins, professor queries, server Ctrl+C, and process cleanup.

```

halit@debian:~/MyHW/Homework6$ ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 6 -t 10
Hogwarts is ready. Port: 5000 | Max Clients: 6 | Timeout: 10s
[2026-05-14 23:24:56] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-14 23:24:56] [SERVER] ENROLL username=hermione type=WIZARD fd=5
[2026-05-14 23:25:00] [SERVER] BREW wizard=hermione ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105
[2026-05-14 23:25:08] [SERVER] BREW wizard=hermione ingredient=DRAGON_SCALE qty=2 old_qty=50 new_qty=52
[2026-05-14 23:25:23] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-14 23:25:23] [SERVER] ENROLL username=ron type=WIZARD fd=6
[2026-05-14 23:25:24] [SERVER] TIMEOUT username=hermione fd=5 elapsed=10s
[2026-05-14 23:25:24] [SERVER] CLIENT_DISCONNECTED username=hermione reason=timeout
[2026-05-14 23:25:26] [SERVER] BREW wizard=ron ingredient=UNICORN_HAIR qty=5 old_qty=20 new_qty=25
[2026-05-14 23:25:32] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-14 23:25:32] [SERVER] ENROLL username=dumbledore type=PROFESSOR fd=5
[2026-05-14 23:25:35] [SERVER] SCROLL professor=dumbledore ingredients=5
[2026-05-14 23:25:36] [SERVER] TIMEOUT username=ron fd=6 elapsed=10s
[2026-05-14 23:25:36] [SERVER] CLIENT_DISCONNECTED username=ron reason=timeout
[2026-05-14 23:25:41] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-14 23:25:41] [SERVER] ENROLL username=mcgonagall type=PROFESSOR fd=6
[2026-05-14 23:25:44] [SERVER] ROSTER professor=mcgonagall clients=2
[2026-05-14 23:25:45] [SERVER] TIMEOUT username=dumbledore fd=5 elapsed=10s
[2026-05-14 23:25:45] [SERVER] CLIENT_DISCONNECTED username=dumbledore reason=timeout
^C[2026-05-14 23:25:50] [SERVER] SHUTDOWN signal=SIGINT
[2026-05-14 23:25:50] [SERVER] CLEANUP_DONE clients=0
halit@debian:~/MyHW/Homework6$ ps aux | grep hogwarts
cat server.log
halit 11462 0.0 0.0 6240 644 pts/5 S+ 23:25 0:00 grep hogwarts
[2026-05-14 23:24:52] [SERVER] SERVER_STARTED port=5000 max_clients=6 timeout=10 ingredients=5
[2026-05-14 23:24:56] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-14 23:24:56] [SERVER] ENROLL username=hermione type=WIZARD fd=5
[2026-05-14 23:25:00] [SERVER] BREW wizard=hermione ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105
[2026-05-14 23:25:08] [SERVER] BREW wizard=hermione ingredient=DRAGON_SCALE qty=2 old_qty=50 new_qty=52
[2026-05-14 23:25:23] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-14 23:25:23] [SERVER] ENROLL username=ron type=WIZARD fd=6
[2026-05-14 23:25:24] [SERVER] TIMEOUT username=hermione fd=5 elapsed=10s
[2026-05-14 23:25:24] [SERVER] CLIENT_DISCONNECTED username=hermione reason=timeout
[2026-05-14 23:25:26] [SERVER] BREW wizard=ron ingredient=UNICORN_HAIR qty=5 old_qty=20 new_qty=25
[2026-05-14 23:25:32] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-14 23:25:32] [SERVER] ENROLL username=dumbledore type=PROFESSOR fd=5
[2026-05-14 23:25:35] [SERVER] SCROLL professor=dumbledore ingredients=5
[2026-05-14 23:25:36] [SERVER] TIMEOUT username=ron fd=6 elapsed=10s
[2026-05-14 23:25:36] [SERVER] CLIENT_DISCONNECTED username=ron reason=timeout
[2026-05-14 23:25:41] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-14 23:25:41] [SERVER] ENROLL username=mcgonagall type=PROFESSOR fd=6
[2026-05-14 23:25:44] [SERVER] ROSTER professor=mcgonagall clients=2
[2026-05-14 23:25:45] [SERVER] TIMEOUT username=dumbledore fd=5 elapsed=10s
[2026-05-14 23:25:45] [SERVER] CLIENT_DISCONNECTED username=dumbledore reason=timeout
[2026-05-14 23:25:50] [SERVER] SHUTDOWN signal=SIGINT
[2026-05-14 23:25:50] [SERVER] CLEANUP_DONE clients=0
halit@debian:~/MyHW/Homework6$

```

Figure 20: Scenario 4 server terminal and log


```

halit@debian:~/MyHW/Homework6$ ./wizard 127.0.0.1 5000 hermione
[WIZARD hermione] CONNECTED server=127.0.0.1:5000
[WIZARD hermione] RECEIVED OK ENROLL hermione
BREW MOONSTONE 5
[WIZARD hermione] SENT BREW MOONSTONE 5
[WIZARD hermione] RECEIVED OK BREW MOONSTONE 5 105
SPELLBOOK
[WIZARD hermione] SENT SPELLBOOK
[WIZARD hermione] RECEIVED OK SPELLBOOK MOONSTONE:5
BREW DRAGON_SCALE 2
[WIZARD hermione] SENT BREW DRAGON_SCALE 2
[WIZARD hermione] RECEIVED OK BREW DRAGON_SCALE 2 52
SPELLBOOK
[WIZARD hermione] SENT SPELLBOOK
[WIZARD hermione] RECEIVED OK SPELLBOOK MOONSTONE:5,DRAGON_SCALE:2
[WIZARD hermione] RECEIVED TIMEOUT DISCONNECT
[WIZARD hermione] DISCONNECTED reason=timeout
halit@debian:~/MyHW/Homework6$

```

Figure 21: Scenario 4 active wizard Hermione terminal

```

halit@debian:~/MyHW/Homework6$ ./wizard 127.0.0.1 5000 harry
[WIZARD harry] CONNECTED server=127.0.0.1:5000
[WIZARD harry] RECEIVED OK ENROLL harry
[WIZARD harry] RECEIVED TIMEOUT DISCONNECT
[WIZARD harry] DISCONNECTED reason=timeout
halit@debian:~/MyHW/Homework6$ ./wizard 127.0.0.1 5000 harry

```

Figure 22: Scenario 4 idle wizard Harry timeout terminal

```

halit@debian:~/MyHW/Homework6$ ./wizard 127.0.0.1 5000 ron
[WIZARD ron] CONNECTED server=127.0.0.1:5000
[WIZARD ron] RECEIVED OK ENROLL ron
BREW UNICORN_HAIR 5
[WIZARD ron] SENT BREW UNICORN_HAIR 5
[WIZARD ron] RECEIVED OK BREW UNICORN_HAIR 5 25
[WIZARD ron] RECEIVED TIMEOUT DISCONNECT
[WIZARD ron] DISCONNECTED reason=timeout
halit@debian:~/MyHW/Homework6$

```

Figure 23: Scenario 4 wizard Ron terminal

```

halit@debian:~/MyHW/Homework6$ ./professor 127.0.0.1 5000 dumbledore
[PROFESSOR dumbledore] CONNECTED server=127.0.0.1:5000
[PROFESSOR dumbledore] RECEIVED OK ENROLL dumbledore
SCROLL
[PROFESSOR dumbledore] SENT SCROLL
[PROFESSOR dumbledore] RECEIVED OK SCROLL MOONSTONE:105,DRAGON_SCALE:52,PHOENIX_FEATHER:30,MANDRAKE_ROOT:75,UNICORN_HAIR:25
[PROFESSOR dumbledore] RECEIVED TIMEOUT DISCONNECT
[PROFESSOR dumbledore] DISCONNECTED reason=timeout
halit@debian:~/MyHW/Homework6$

```

Figure 24: Scenario 4 professor Dumbledore terminal

```

halit@debian:~/MyHW/Homework6$ ./professor 127.0.0.1 5000 mcgonagall
[PROFESSOR mcgonagall] CONNECTED server=127.0.0.1:5000
[PROFESSOR mcgonagall] RECEIVED OK ENROLL mcgonagall
ROSTER
[PROFESSOR mcgonagall] SENT ROSTER
[PROFESSOR mcgonagall] RECEIVED OK ROSTER dumbledore,mcgonagall
[PROFESSOR mcgonagall] RECEIVED SERVER SHUTDOWN
[PROFESSOR mcgonagall] DISCONNECTED reason=shutdown
halit@debian:~/MyHW/Homework6$

```

Figure 25: Scenario 4 professor McGonagall terminal

11 Challenges

11.1 Handling TCP Lines Correctly

The first important issue was that TCP can split one command into multiple reads or join many commands in one read. So I did not parse directly after every `recv()`. Instead, every client has a line buffer. Bytes are appended until newline. This fixed partial reads and also made `ERR TOOLONG` easier to implement.

11.2 Idle Timeout Without Sleep

Another problem was idle timeout. A simple `sleep()` loop would break the homework rule and also block client processing. I solved it by checking clients before every `select()` call and calculating the nearest remaining timeout. This lets `select()` wake up exactly when the next client may timeout.

11.3 Shared Client Code

Wizard and professor clients are very similar, but their role names are different. I moved the common client logic into `client_app.c`. This avoided duplicated socket and signal handling code. The server still enforces the actual permissions, so the professor program cannot bypass rules.

12 Conclusion

The final implementation builds three programs with the required compiler flags. The server uses one `select()` loop, maintains per-client state, rejects extra clients when full, disconnects idle clients, handles Ctrl+C cleanly, and implements the required protocol responses. The mandatory scenario outputs show the main behavior expected by the homework.